Unidade 03 | Capítulo 01

Acesso a Banco de Dados (JDBC)

Prof.: Allberson Dantas



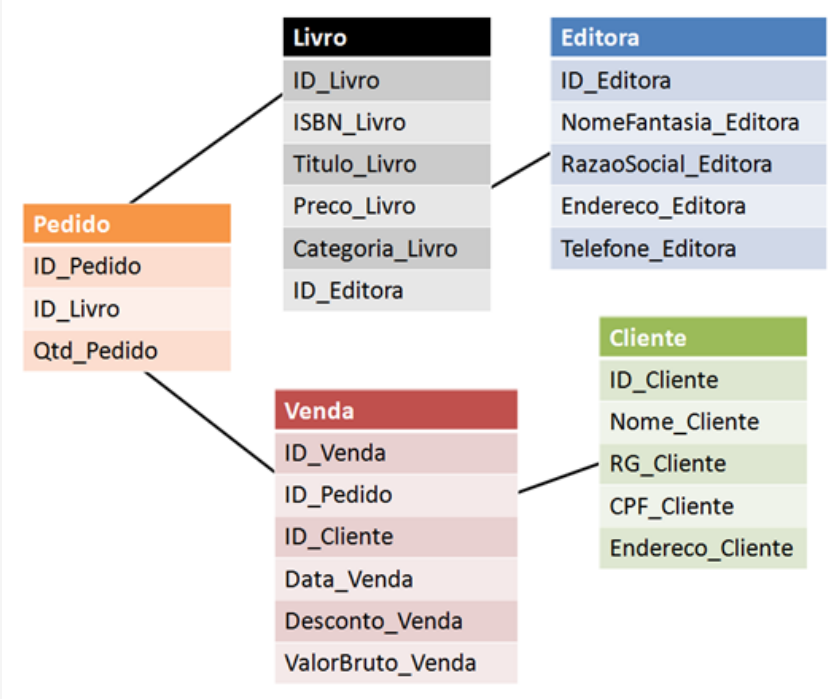


Sumário

- Conceitos básicos de banco de dados relacional;
- JDBC (Java Database Connectivity);
- Configuração do ambiente: driver JDBC, conexão com banco de dados (Ex: MySQL).

O que é um Banco de Dados Relacional?

- O modelo relacional é amplamente utilizado na maioria dos sistemas de informação. Ele se baseia na organização dos dados em tabelas (ou relações), que podem ser visualizadas como planilhas compostas por linhas e colunas.
- Cada tabela armazena dados relacionados a uma entidade (como usuários, produtos, pedidos), e cada linha representa um registro único dessa entidade. As colunas da tabela correspondem aos atributos da entidade — por exemplo: nome, email, idade, etc.



https://economia-inform-intel.blogspot.com/2017/11/bancos-de-dados-relacionais-access-e-sql.html

A Origem do Banco de Dados Relacional

1950-1960:
Modelos Antecessores

Modelos Antecessores:

- **Modelo Hierárquico**: Organiza dados como uma árvore pai-filho. Rígido e de difícil manutenção.
- **Modelo em Rede**: Mais flexível que o hierárquico, exige navegação manual e é complexo de implementar.

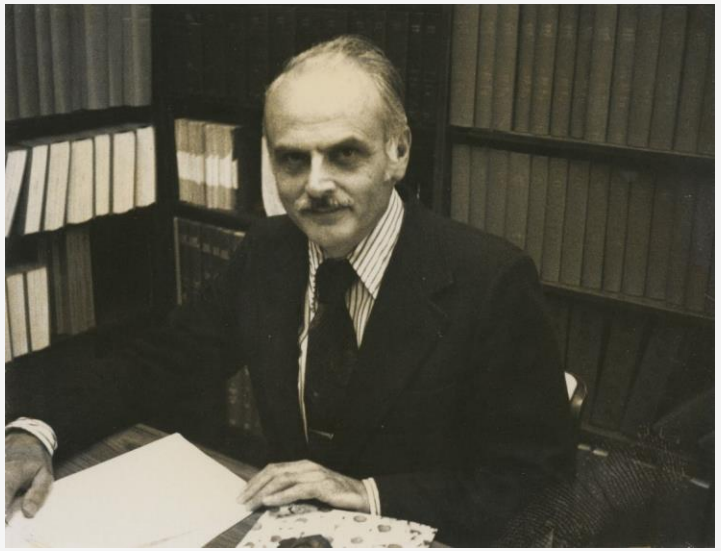
A Origem do Banco de Dados Relacional

1970:
A Revolução de Codd

O surgimento do modelo relacional:
Edgar F. Codd: “A Relational Model of Data for Large Shared Data Banks”.

- Tabelas relacionais (em vez de hierárquicas);
- Chaves primárias e estrangeiras;
- Operações declarativas (como SELECT, JOIN, PROJECT) para manipular dados.

Separação entre lógica e armazenamento físico de dados.



https://www.ibm.com/history/edgar-codd

A Origem do Banco de Dados Relacional

Anos 1970-1980:
Nascimento dos Primeiros SGBDs Relacionais

System R: protótipo criado pela IBM com base no modelo de Codd.
Outros projetos e produtos pioneiros surgiram:
- Ingres (Universidade da Califórnia, Berkeley);
- Oracle (fundada em 1977);
- DB2 (lançado pela IBM em 1983);
- SQL Server (lançado pela Microsoft em 1989).

A Origem do Banco de Dados Relacional

Anos 1990 em diante:
Consolidação e Padrões

Legado Duradouro:
Mais de 50 anos depois, o modelo relacional ainda domina sistemas críticos:

- Sistemas financeiros e bancários;
- Plataforma de e-commerce;
- Sistemas ERP;
- Aplicações corporativas e web.

Chave Primária (Primary Key)

Uma **chave primária** é um campo (ou conjunto de campos) que identifica de forma única cada linha (registro) dentro de uma tabela.

- Deve ser única: dois registros não podem ter o mesmo valor na chave primária.
- Não pode ser nula.
- Exemplos comuns: CPF, matrícula, código do produto, ID numérico.

id_usuario	nome	email
1	Ana Souza	ana@email.com
2	João Lima	joao@email.com

Chave Estrangeira (Foreign Key)

Uma **chave estrangeira** é um campo em uma tabela que faz referência à chave primária de outra tabela. Esse relacionamento conecta informações entre entidades diferentes, mantendo a integridade dos dados.

- Estabelece vínculos lógicos entre as tabelas.
- Garante que os dados relacionados existam no banco (integridade referencial).

id_pedido	Id_usuario	total
1	1	89.90
2	2	150.00

O que é SQL?

SQL (Structured Query Language) é a linguagem padrão para gerenciar e manipular dados em sistemas de bancos relacionais (como MySQL, PostgreSQL, Oracle, SQL Server, entre outros). É através dessa linguagem que realizamos operações como:

- Criar tabelas e estruturas (CREATE);
- Inserir dados (INSERT);
- Consultar dados (SELECT);
- Atualizar dados (UPDATE);
- Remover registros (DELETE).

Exemplo com SQL

Vamos primeiro criar uma tabela simples chamada usuarios, que armazena informações sobre usuários de um sistema:

```sql
CREATE TABLE usuarios (
    id_usuario INT PRIMARY KEY,
    nome VARCHAR(100),
    email VARCHAR(100)
);
```

Exemplo com SQL

Com a tabela criada, podemos adicionar dados com o comando INSERT:

```sql
INSERT INTO usuarios (id_usuario, nome, email)
VALUES (1, 'Ana Souza', 'ana@email.com');

INSERT INTO usuarios (id_usuario, nome, email)
VALUES (2, 'João Lima', 'joao@email.com');
```



```
1|Ana Souza|ana@email.com
2|João Lima|joao@email.com
```

Exemplo com SQL

UPDATE: Agora, suponha que Ana mudou de e-mail. Podemos atualizar o registro assim:

```sql
UPDATE usuarios
SET email = 'ana.souza@novaempresa.com'
WHERE id_usuario = 1;
```

```
1|Ana Souza|ana.souza@novaempresa.com
2|João Lima|joao@email.com
```

Exemplo com SQL

DELETE: Agora vamos remover o registro do usuário João:

```
DELETE FROM usuarios
WHERE id_usuario = 2;
```



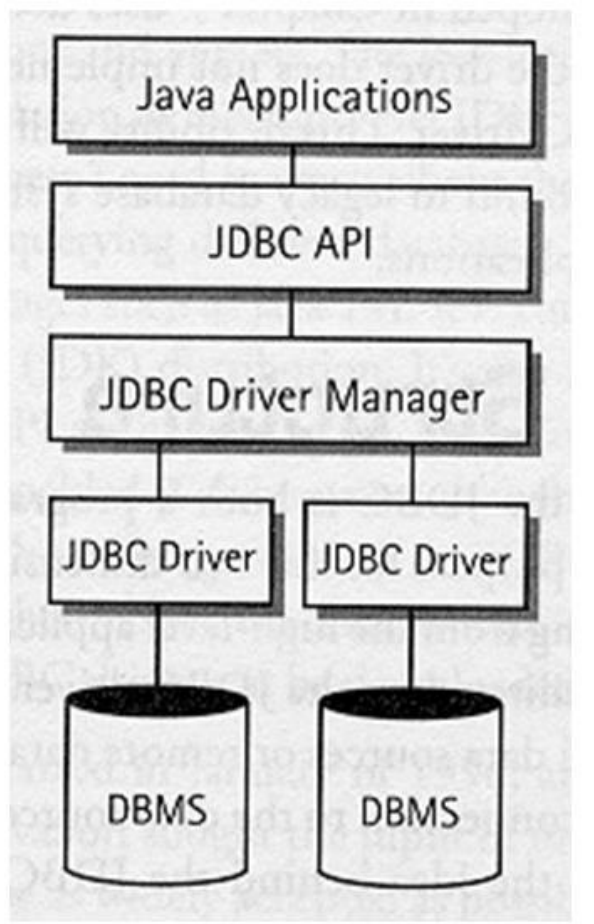
```
1|Ana Souza|ana.souza@novaempresa.com
```

O que é JDBC?

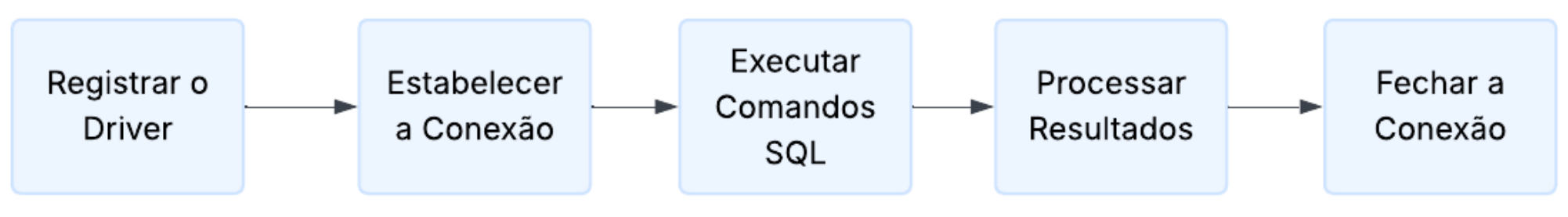
JDBC é uma da linguagem Java que permite a comunicação entre aplicações Java e bancos de dados relacionais. Sua principal função é fornecer os meios para que a aplicação envie comandos SQL ao banco de dados e receba os resultados dessas operações.

Por meio do JDBC, o desenvolvedor pode:

- Estabelecer uma conexão com o banco de dados;
- Executar instruções SQL diretamente a partir do Java;
- Receber os resultados de consultas (como registros de uma tabela);
- Tratar erros, exceções e transações



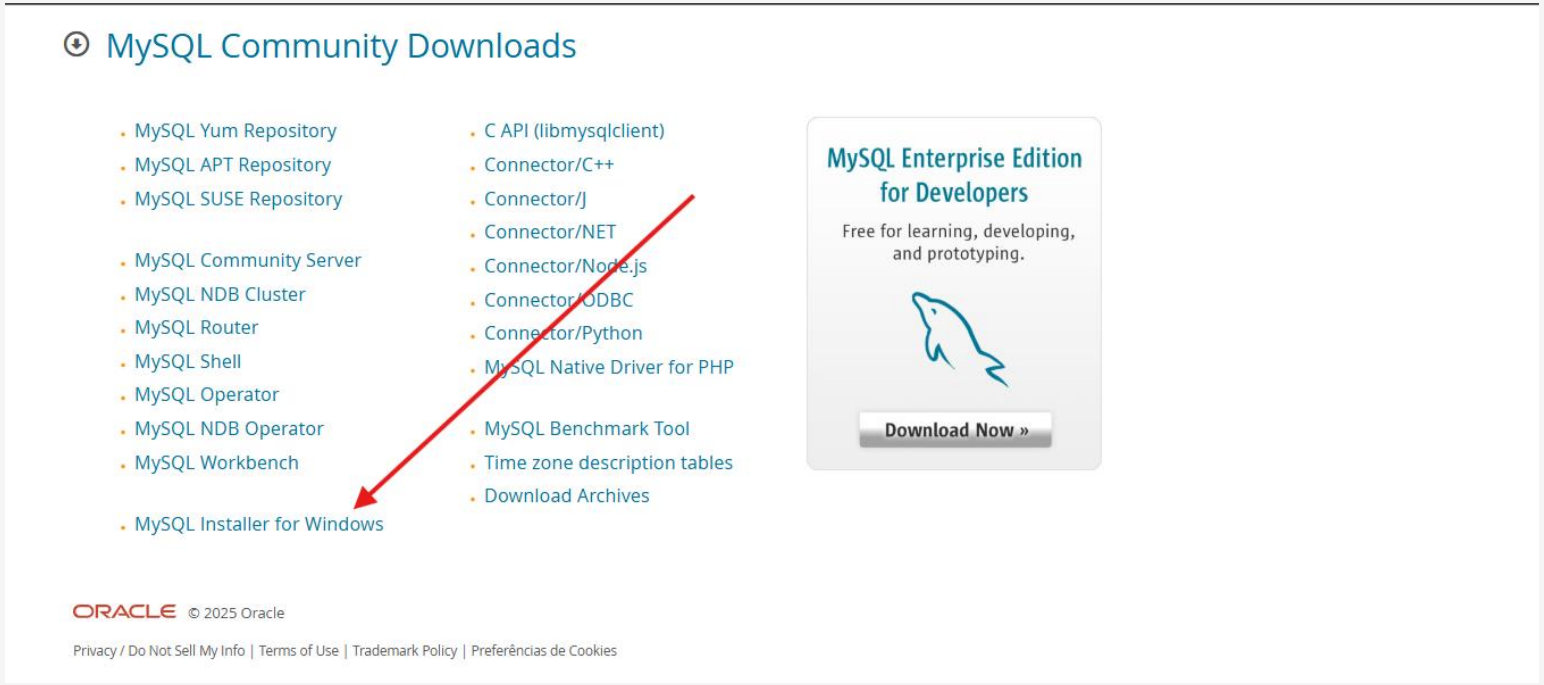
Ciclo de Conexão com JDBC



Instalação MySQL

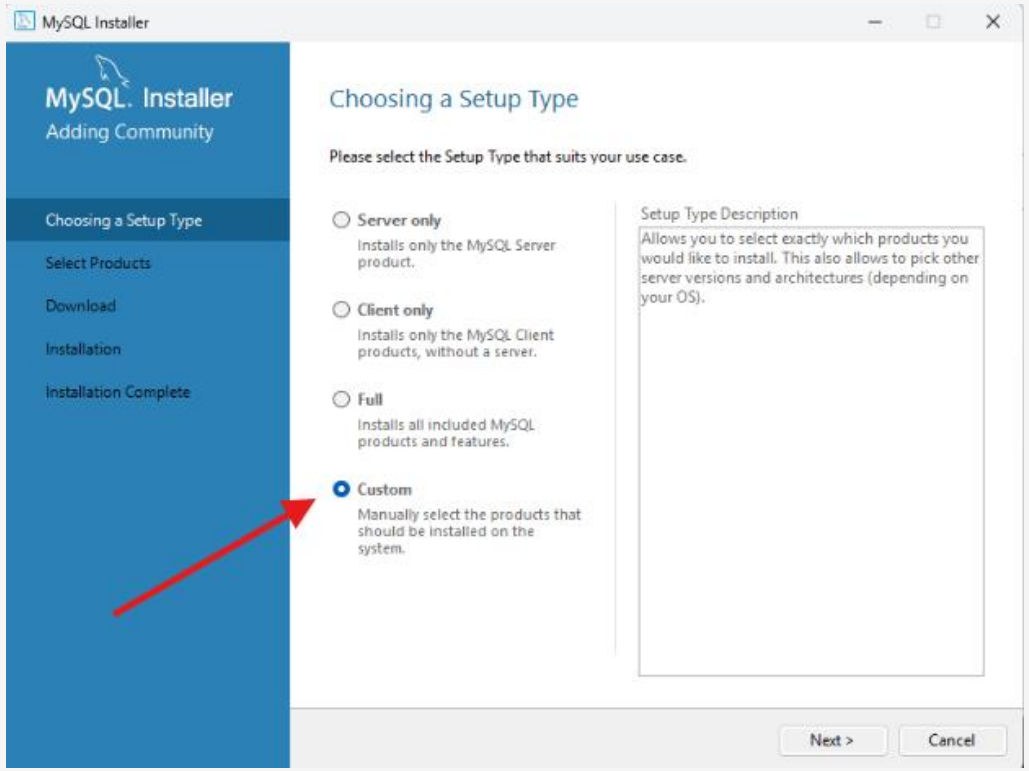
Para utilizar o JDBC precisamos instalar o MySQL, siga os seguintes passos:

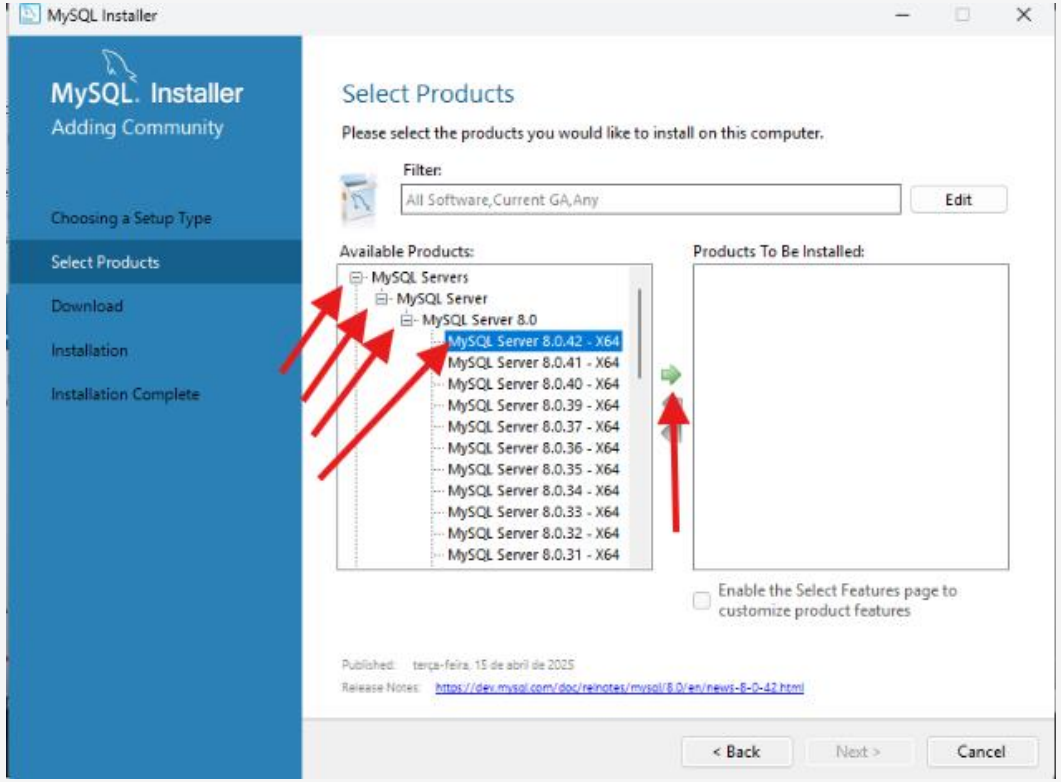
Inicialmente, baixe o instalador em: https://dev.mysql.com/downloads/



Instalação MySQL

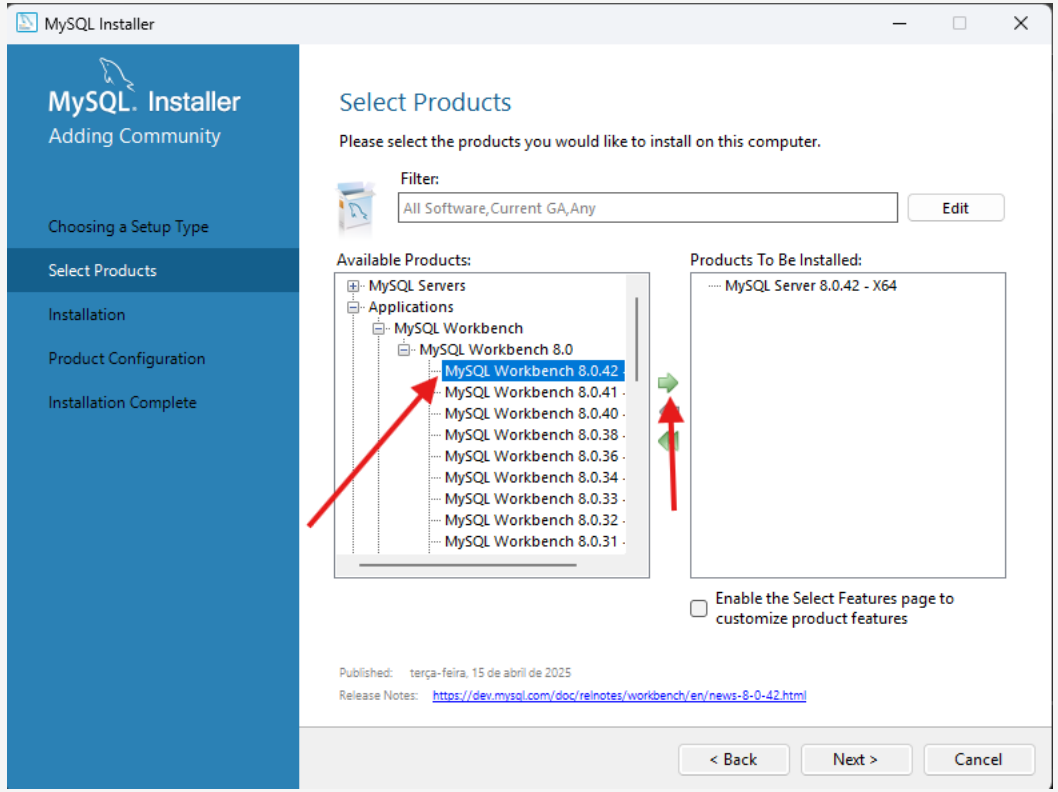
Siga o passo a passo:

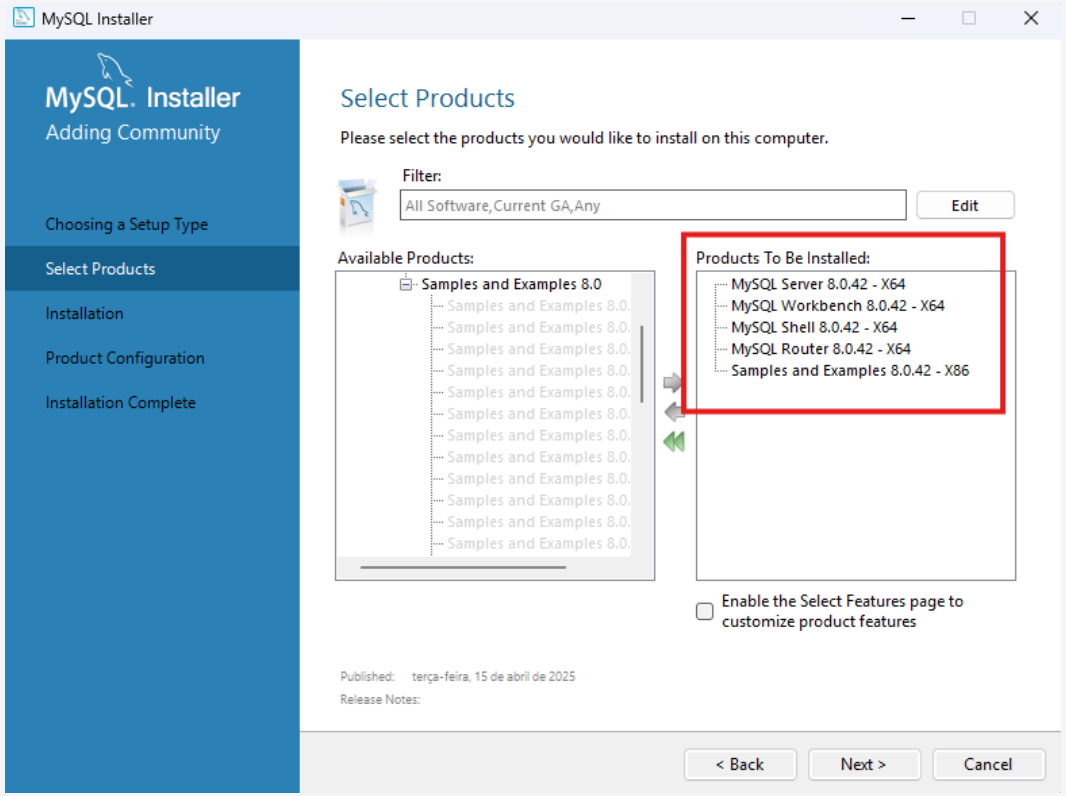




Instalação MySQL

Siga o passo a passo:

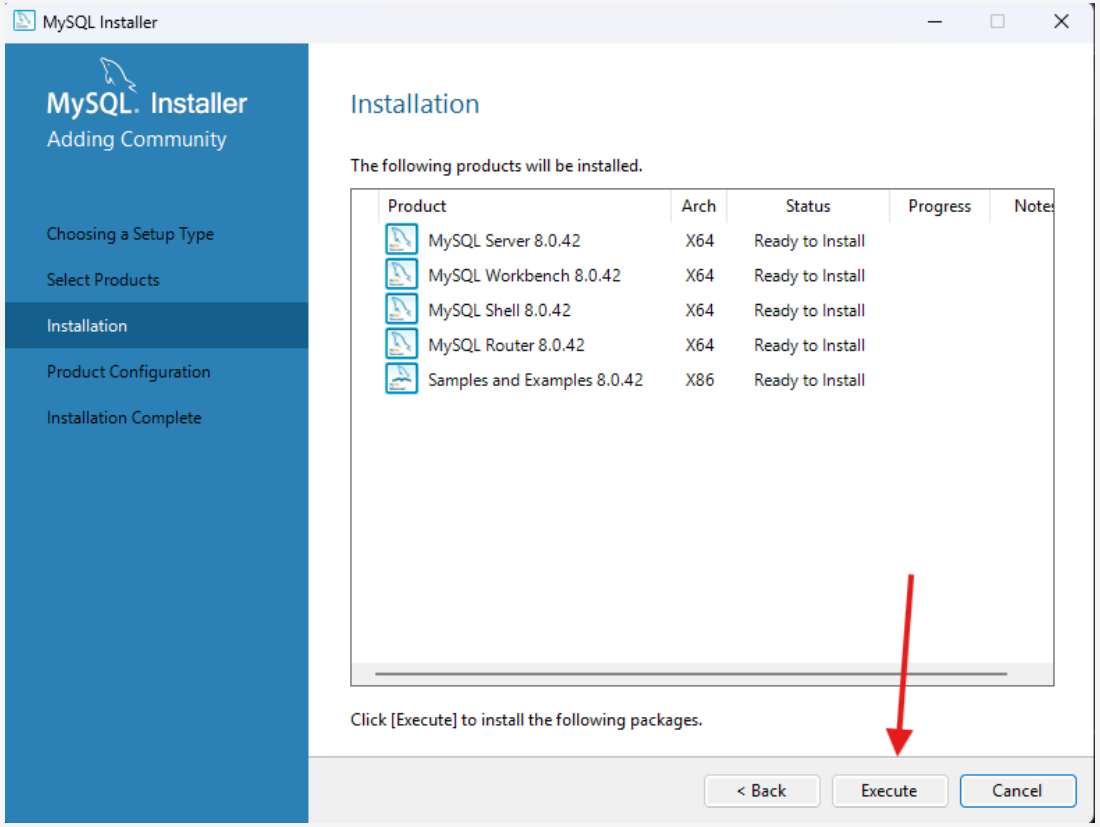


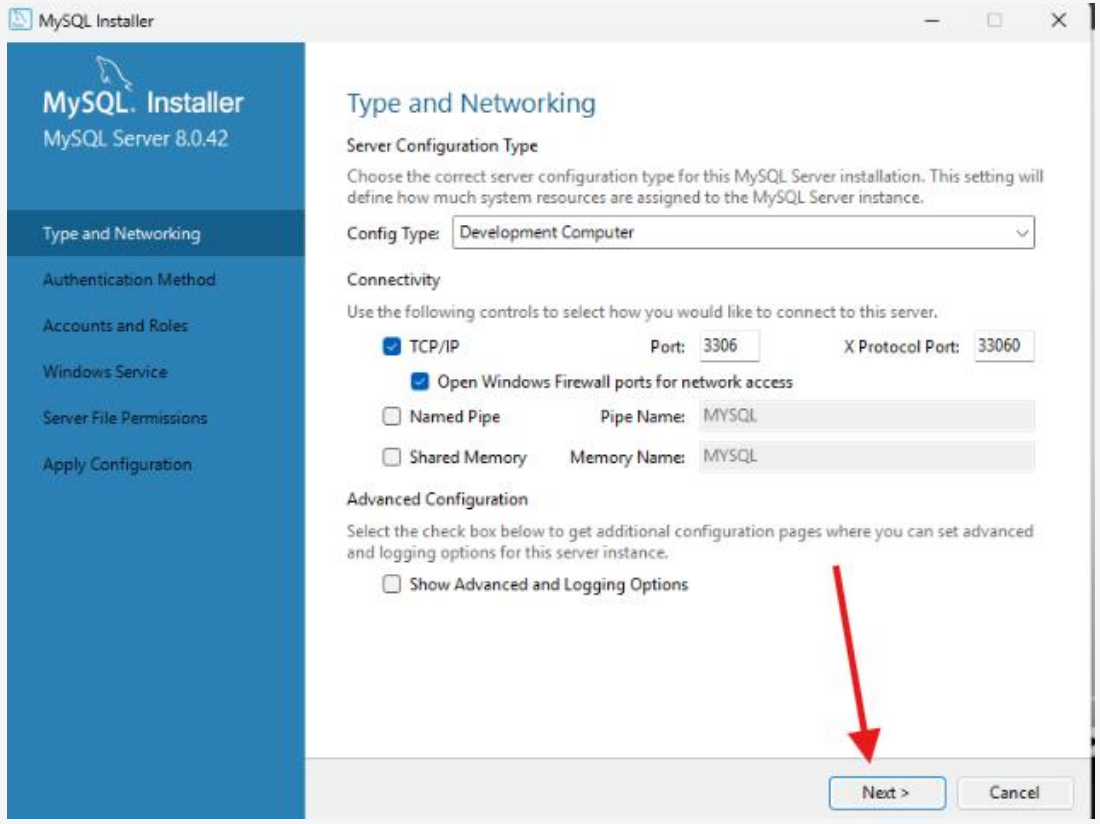


CAPACITA iRede

Instalação MySQL

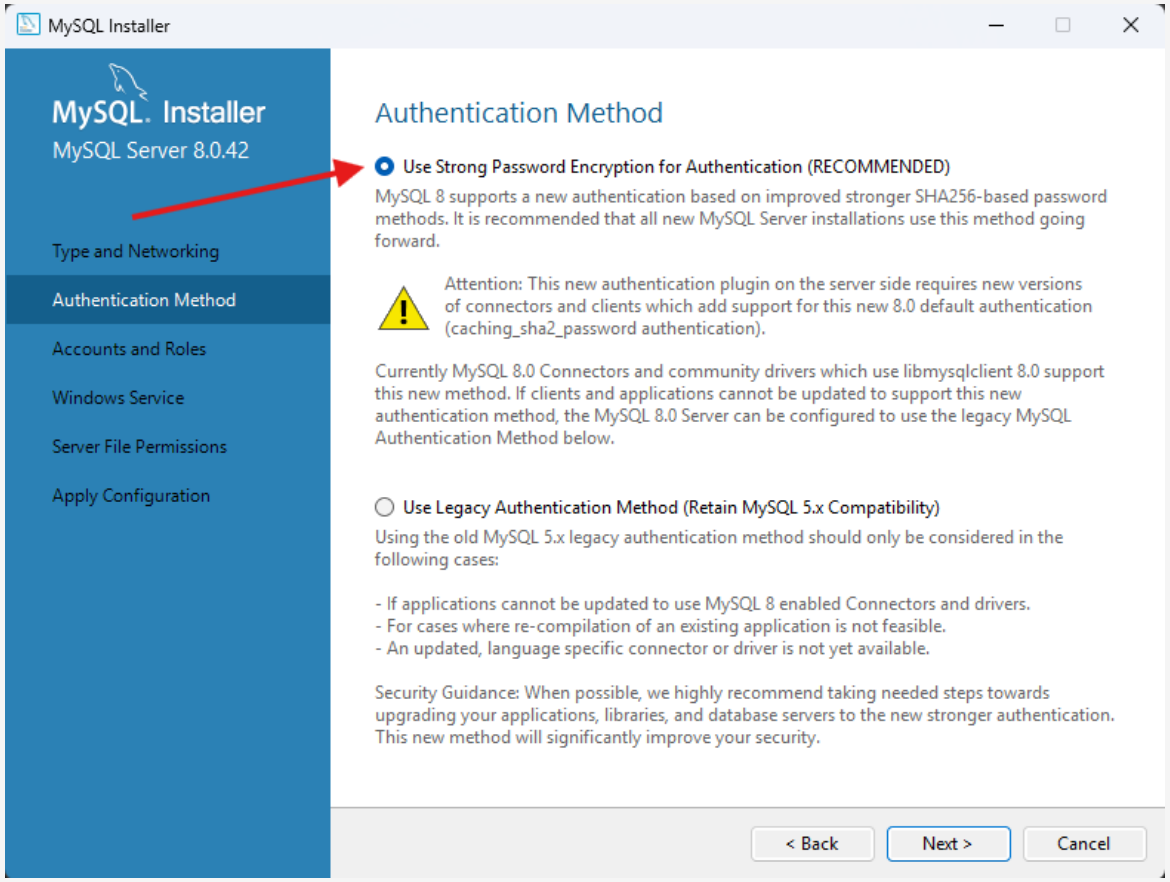
Siga o passo a passo:





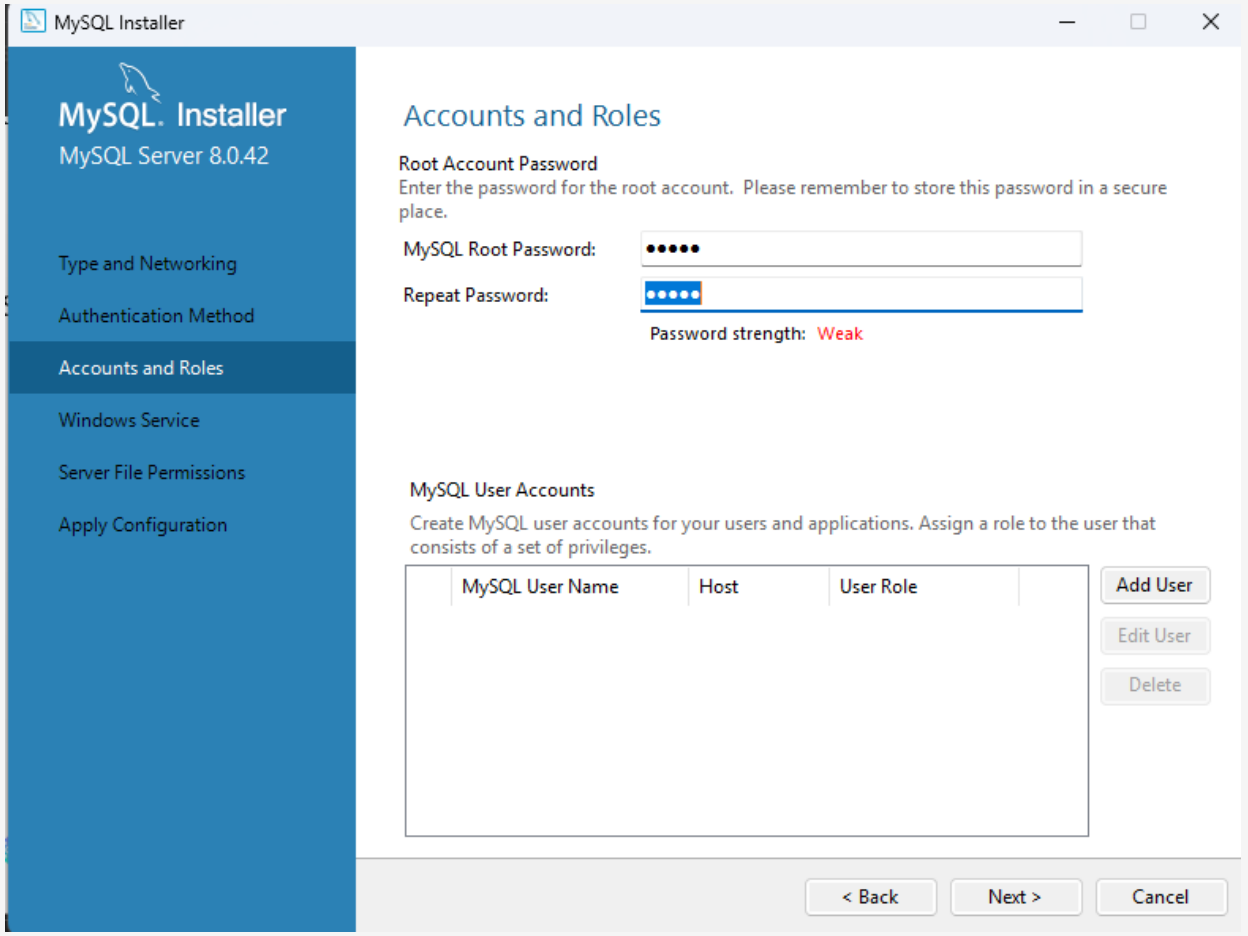
Instalação MySQL

Siga o passo a passo:



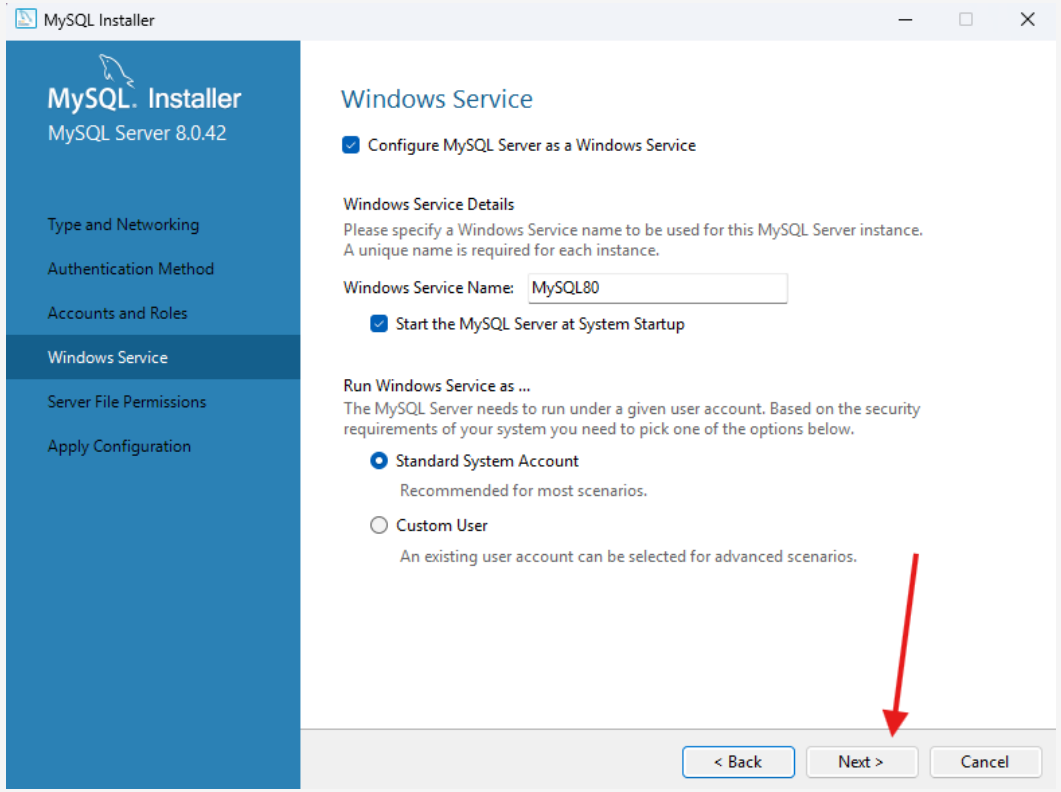
Instalação MySQL

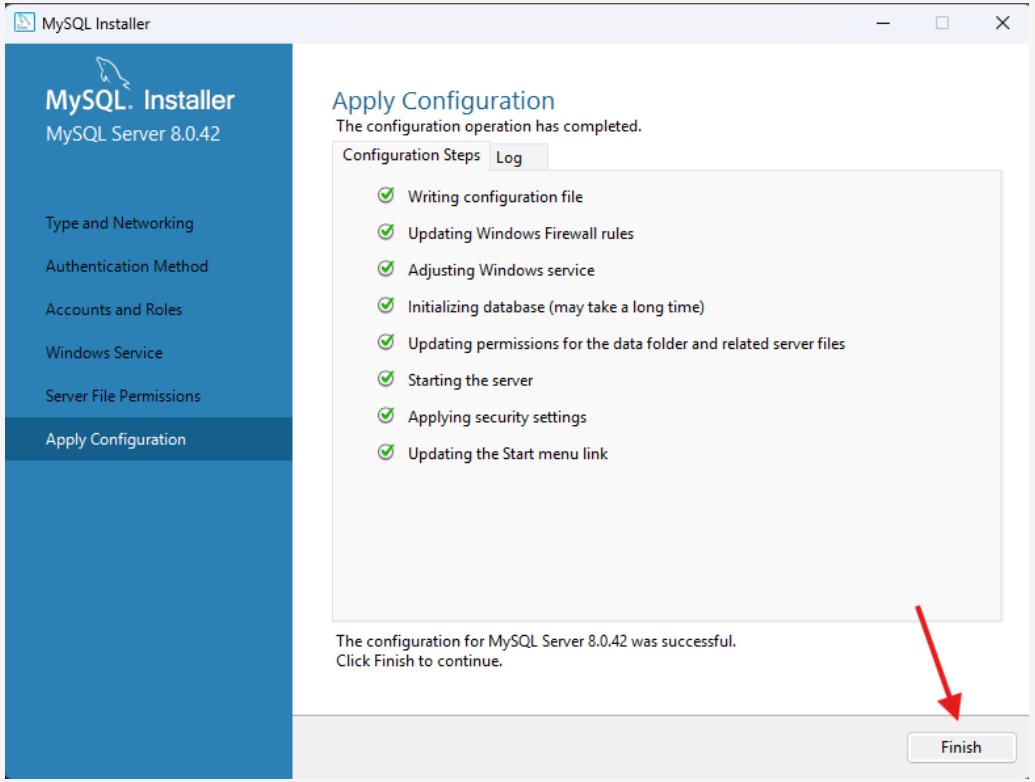
Siga o passo a passo (Crie sua senha para ROOT):



Instalação MySQL

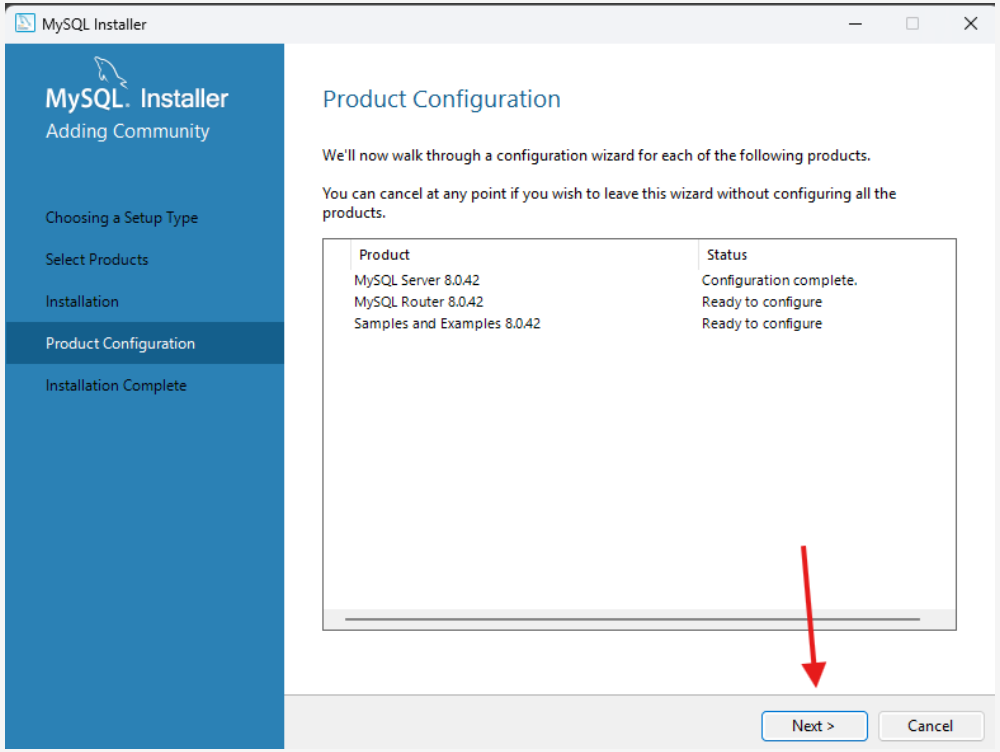
Siga o passo a passo:

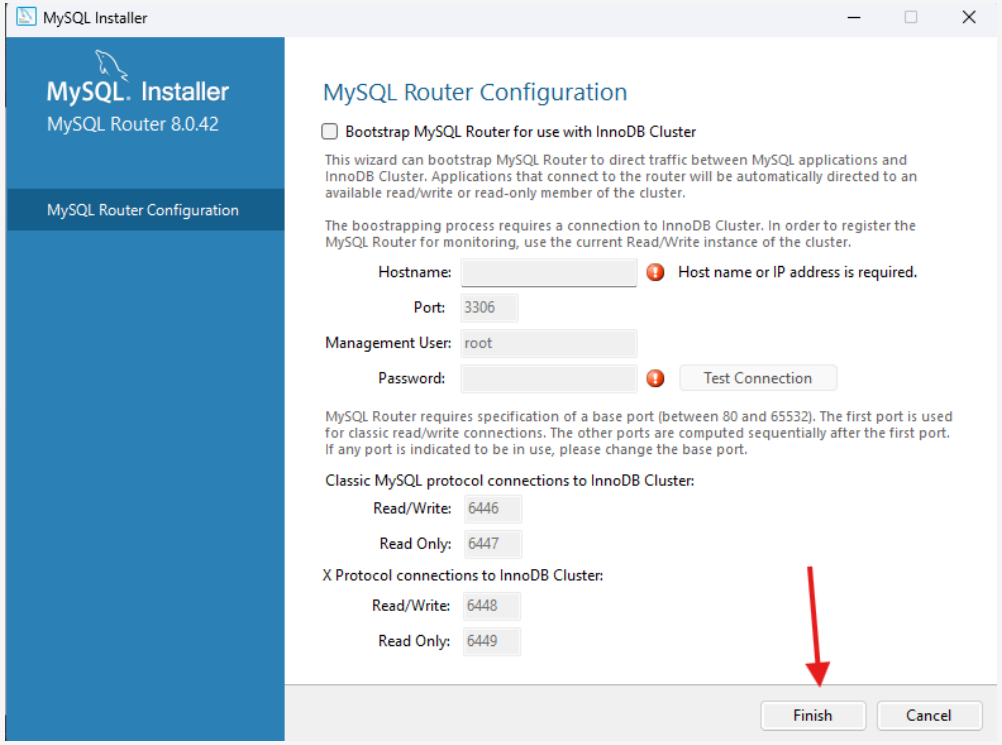




Instalação MySQL

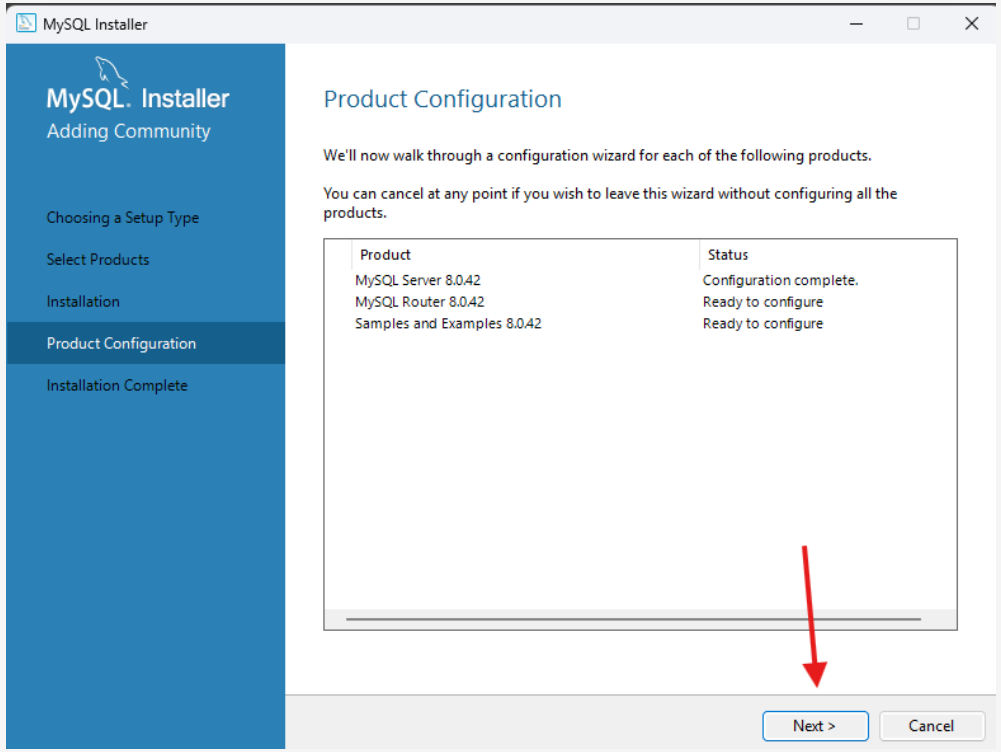
Siga o passo a passo:

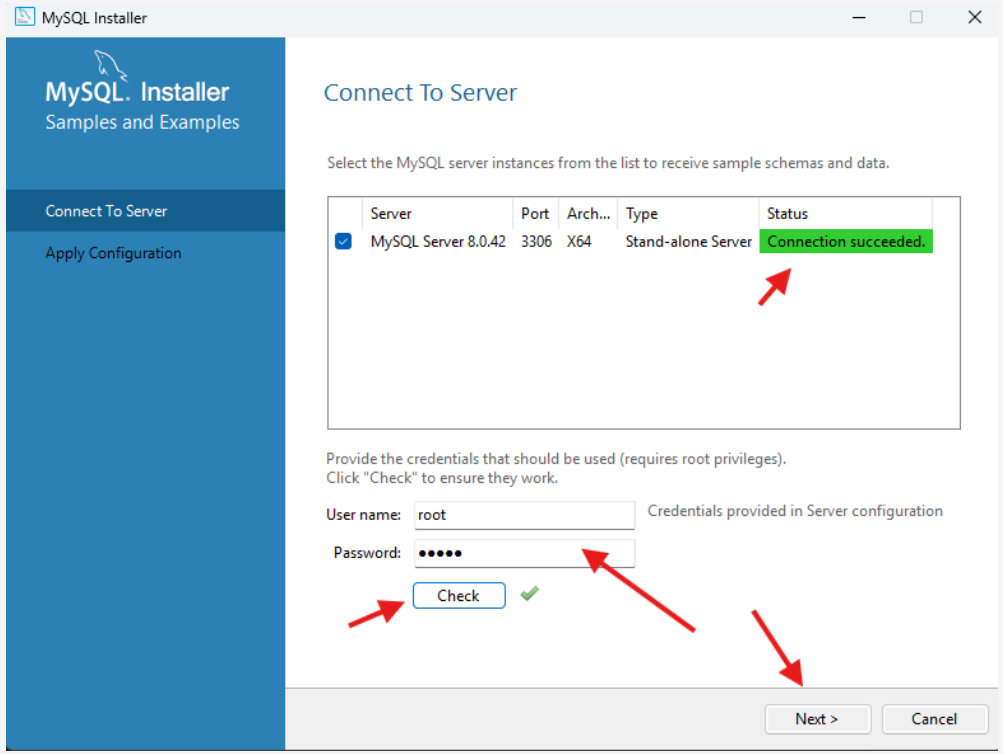




Instalação MySQL

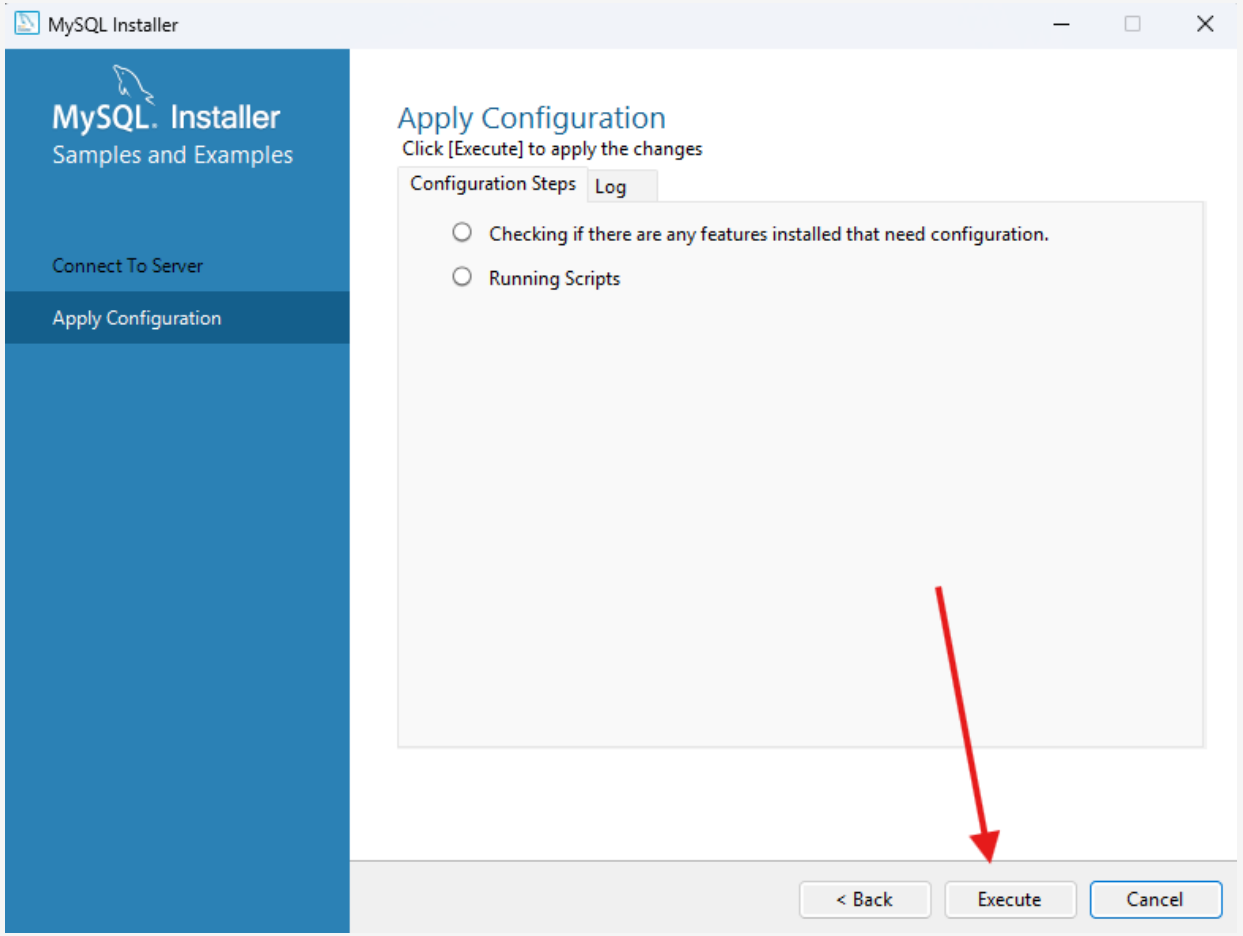
Siga o passo a passo:





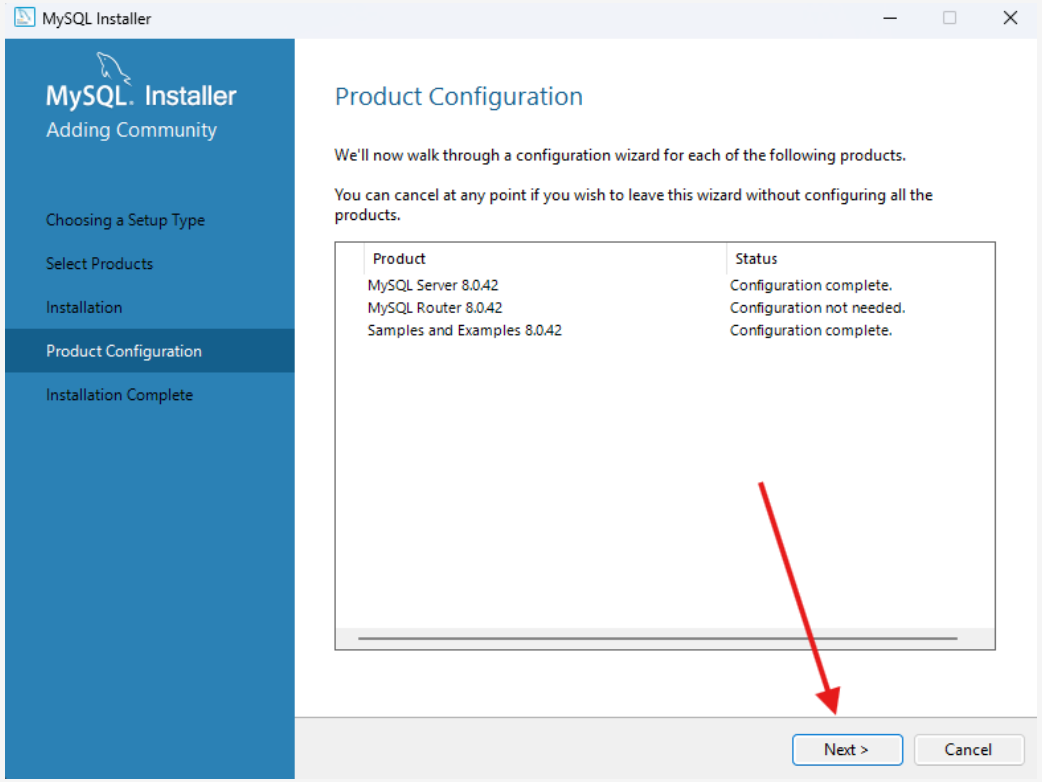
Instalação MySQL

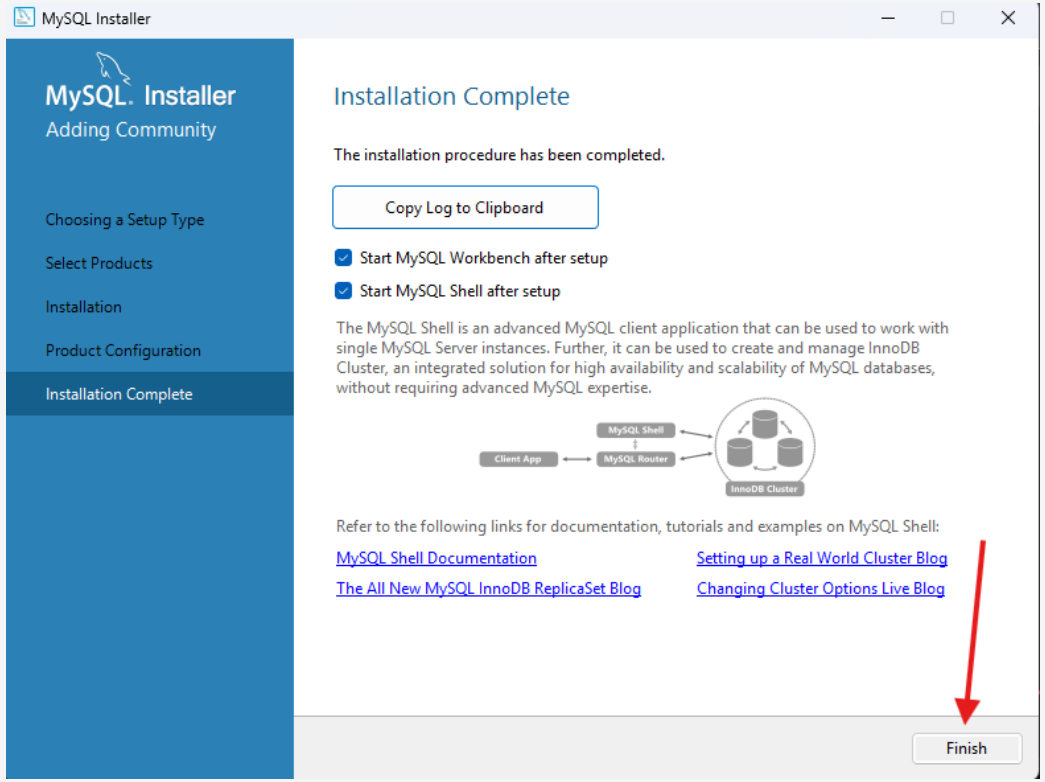
Siga o passo a passo:



Instalação MySQL

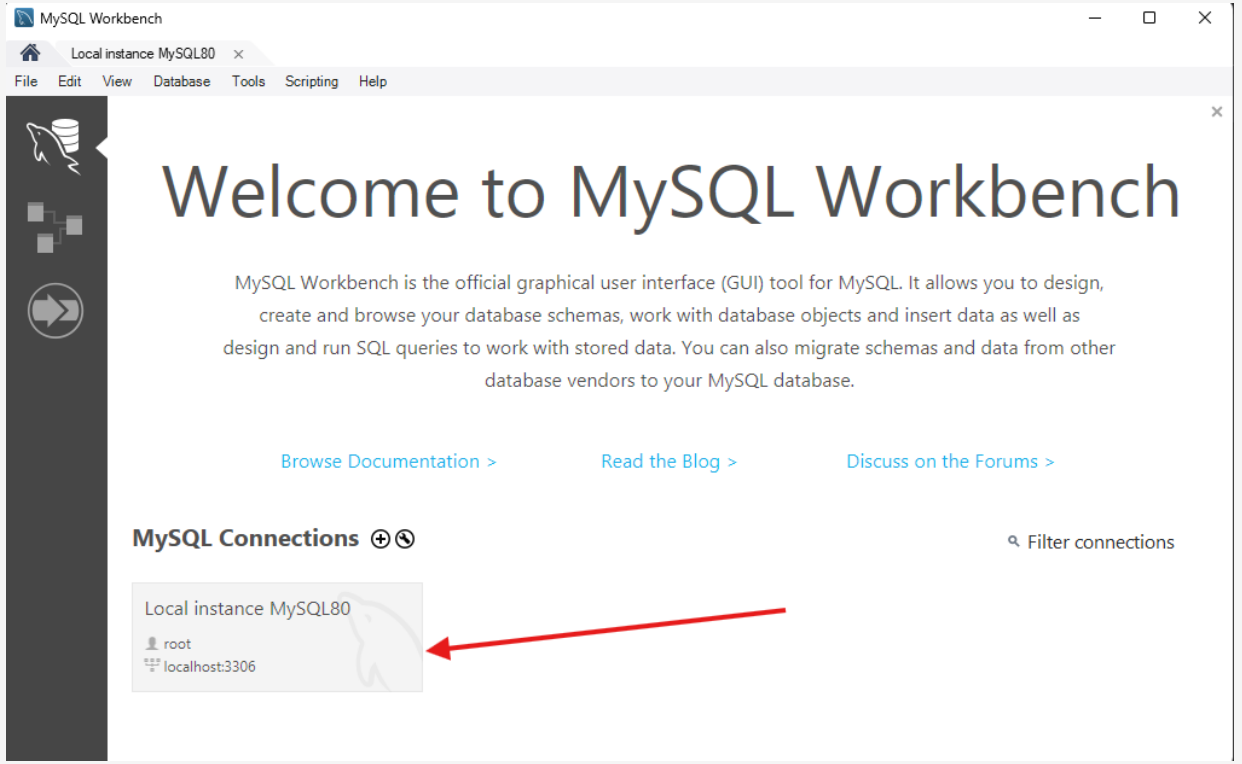
Siga o passo a passo:





Criando Banco de Dados com MySQL

Com o MySQL instalado, vamos agora criar nosso Banco de Dados, entrem no MySQL Workbench:



Criando Banco de Dados com MySQL

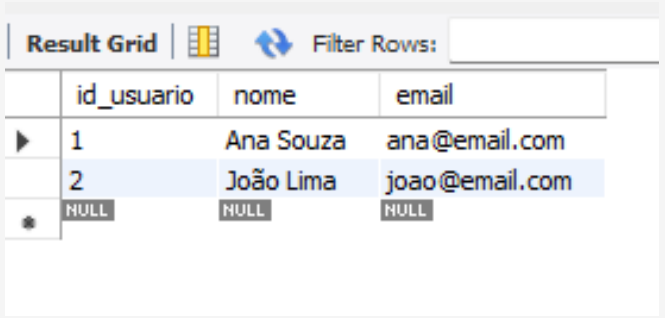
Após entrar no Workbench, criamos nosso banco e fazemos operações de inserção:

```sql
CREATE DATABASE meubanco;
USE meubanco;

CREATE TABLE usuarios (
    id_usuario INT PRIMARY KEY,
    nome VARCHAR(100),
    email VARCHAR(100)
);

INSERT INTO usuarios VALUES
(1, 'Ana Souza', 'ana@email.com'),
(2, 'João Lima', 'joao@email.com');

select * from usuarios
```



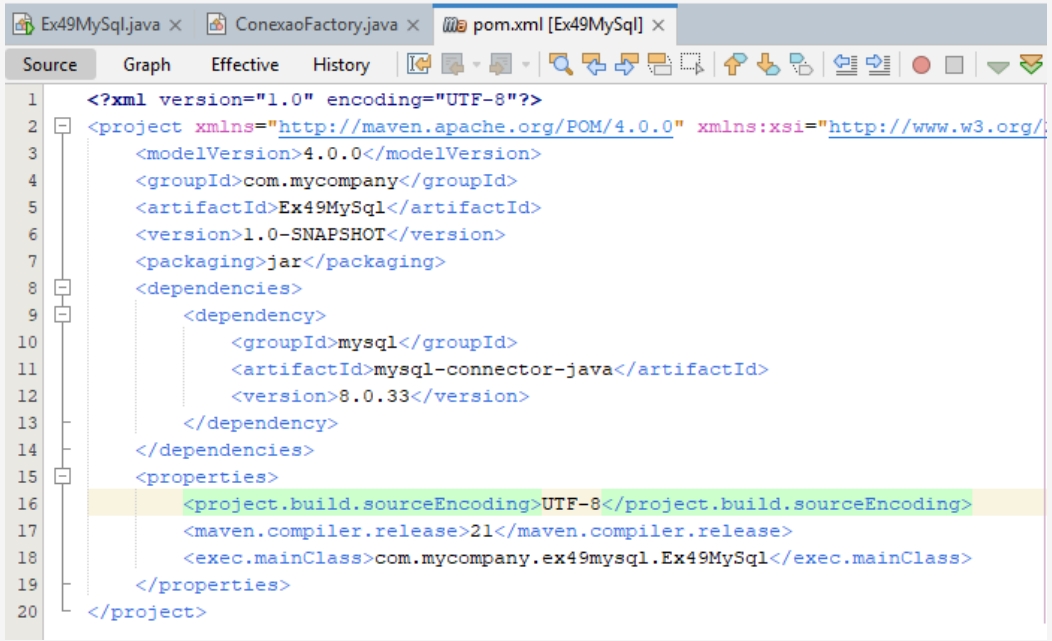
Conectando Java ao BD via JDBC

Agora, crie um projeto Java com Maven normalmente:

Após criar o projeto vá em projects > no seu projeto atual: Project Files > pom.xml

Edite o código adicionando o driver para usar MySQL:

```
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.33</version>
</dependency>
```



Material de Apoio: Para rodar o código, execute o projeto da pasta Exemplos: Ex49MySql

Conectando Java ao BD via JDBC

Com o projeto estruturado, criamos a classe ConexaoFactory:

```
package com.mycompany.ex49mysql;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexaoFactory {
    private static final String URL = "jdbc:mysql://localhost:3306/meubanco";
    private static final String USUARIO = "root";
    private static final String SENHA = "sua_senha";

    public static Connection conectar() throws SQLException {
        return DriverManager.getConnection(url:URL, user:USUARIO, password:SENHA);
    }
}
```

Observação: Substitua “sua_senha” pela senha que você configurou durante a instalação do MySQL.

Material de Apoio: Para rodar o código, execute o projeto da pasta Exemplos: Ex49MySql

Conectando Java ao BD via JDBC

Configuramos o nosso arquivo principal da seguinte forma:

```java
package com.mycompany.ex49mysql;

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.Statement;

public class Ex49MySql {

    public static void main(String[] args) {
        try (Connection conn = ConexaoFactory.conectar()) {
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql:"SELECT * FROM usuarios");

            while (rs.next()) {
                System.out.println(
                    rs.getInt(columnLabel:"id_usuario") + " - " +
                    rs.getString(columnLabel:"nome") + " - " +
                    rs.getString(columnLabel:"email")
                );
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
Compiling 2 source files with javac [debug release 21] to target\classes

--- exec:3.1.0:exec (default-cli) @ Ex49MySql ---
1 - Ana Souza - ana@email.com
2 - Jo◆o Lima - joao@email.com
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Material de Apoio: Para rodar o código, execute o projeto da pasta Exemplos: Ex49MySql

Boas Práticas com o JDBC

Trabalhar com JDBC exige não apenas escrever código funcional, mas também aplicar boas práticas que melhoram a segurança, a legibilidade e a manutenção do projeto. Como:

- **Sempre feche conexões, statements e result sets**: Todo recurso aberto com o banco consome memória e mantém portas abertas. Quando você não os fecha, isso pode causar vazamentos de memória e sobrecarregar o sistema com conexões abandonadas;
- **Use PreparedStatement para evitar SQL Injection**: O PreparedStatement evita que valores maliciosos inseridos pelo usuário quebrem a lógica SQL da aplicação;
- **Centralize a lógica de conexão**: Manter toda a lógica de conexão concentrada em uma única classe.



Lista de Exercícios e Tarefas

Responda ao Quiz do Capítulo.

Desenvolva as questões da TAREFA 16 – Acesso a Banco de Dados (JDBC)

CAPACITA iRede

INOVAÇÃO E TECNOLOGIA

RESIDÊNCIA EM TIC 20



EXECUTOR

iRede
INOVAÇÃO E TECNOLOGIA

INICIATIVA

MCTI FUTURO

COORDENADORA PPI

Softex

MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO

*Este projeto foi apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991, no âmbito do PPI-SOFTEX, coordenado pela Softex e publicado Residência em TIC 20, DOU 01245.002631/2023-58.